Lecture – 9: Evaluating AI Agents

Building Systems That Measure
What Actually Matters

Agentic AI @ UIUC | April 2026

Agenda

01	Why This Matters	3 min
02	What Is an Agent Eval System?	4 min
03	Types of Agent Evaluations	10 min
04	Key Metrics That Matter	4 min
05	Real Tools and Systems	5 min
06	Challenges and How to Design Evals	5 min
07	Closing Takeaway	2 min

30-minute presentation

Agents Are Powerful… But Unreliable

We've moved from models that answer questions to agents that take actions.

Make Decisions

Autonomous reasoning across multiple steps

Use Tools

APIs, databases, external services

Operate Over Time

Long-running tasks with state

What Is an Agent Eval System?

A framework to measure whether an agent completes tasks correctly, efficiently, and reliably.

- Not just "was the answer right?"
- It's "did the agent behave correctly?"
- Covers the full trajectory, not a single output

Traditional ML vs Agent Evaluation

	Traditional ML	AI Agents
Output	Static predictions	Multi-step actions
Metric	Accuracy / F1	Task success rate
Scope	Single prediction	Full trajectory
Env	Fixed dataset	Dynamic world

Agents require evaluation of behavior, not just correctness.

Types of Agent Evaluations

4 core evaluation approaches

There is way too many of them!!!!

01 Outcome-Based Evaluation

Did the agent succeed at the task?

- Deploy an application successfully
- Answer a user question correctly
- Fix a bug and pass tests

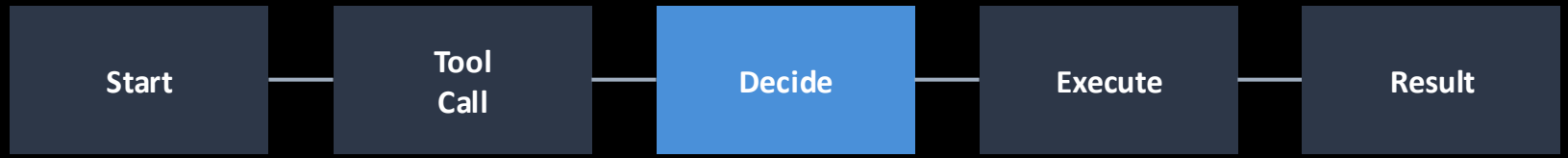
Key Metrics

Success Rate
Binary pass/fail per task

Time to Completion
End-to-end latency

02 Trajectory / Step Evaluation

Evaluate each decision the agent makes along the way.



Tool selection	Did it pick the right tool?
Number of steps	Was the path efficient?
Error handling	Did it recover from failures?

03 LLM-as-Judge

Use another model to grade agent outputs.

Use Cases

- Quality assessment
- Reasoning evaluation
- Helpfulness scoring
- Safety & alignment checks

Tradeoffs

- ✅ Fast & scalable
- ✅ No human labels needed
- ⚠️ Imperfect — biases exist
- ⚠️ Needs calibration

04 Simulation Environments

Test agents in controlled, reproducible settings.

Fake APIs

Mock endpoints that return deterministic responses

Sandbox Cloud

Isolated environments for infra agents to operate safely

Scripted Users

Simulated user interactions with predefined behaviors

Key Metrics That Matter

95%	< 2s	$0.03	3	99%
Success Rate	Latency	Cost per Task	Steps (Efficiency)	Reliability

Which Agent Is Better?

Task: Deploy a web application

Agent A

95%

success rate

3 steps avg

Agent B

95%

success rate

12 steps avg

Real Tools & Systems

What teams actually use in practice

Tooling Landscape

LangSmith

Tracing, logging,
dataset management,
eval suites

Langfuse

Open Source LLM observability platform

DATADOG

Enterprise Platform with observability

Logging | Tracing | Replay | Metrics Dashboards

Current Reality

"Most serious teams build custom eval systems."

Reason

- Domain-specific tasks require domain-specific evaluation
- Custom success criteria that off-the-shelf tools can't capture
- Internal benchmarks aligned with business outcomes
- Integration with existing CI/CD and deployment pipelines

DEMO

Customer Agent with Langfuse

Agents Evals are really hard!!!!

The unsolved challenges

Core Challenges

Non-Determinism

Same input can produce different outputs each run

Long Task Chains

Errors compound across dozens of sequential steps

No Clear Ground Truth

Multiple valid paths to solving the same problem

Tool Failures

External dependencies introduce uncontrolled variables

"Agents don't just fail — they fail in unpredictable ways."

Designing an Eval System

A 3-layer framework

Layer 3: Production

Real-world monitoring, alerting, drift detection

Layer 2: Scenario

Full end-to-end task evaluation in sandbox

Layer 1: Unit

Individual tool correctness & prompt testing

Start at unit level, expand outward as your agent matures.

The Eval Pipeline

1. Define Task
2. Run Agent
3. Log Steps
4. Score Outcome
5. Analyze Failures

"This is your system design moment — automate the loop."

Real Example: Customer Support AI Agent

Scenario: AI agent answers customer support emails (e.g. Zendesk, Shopify)

Customer Message

"I was charged twice for my order. Can you refund me?"

Agent Response

"Sorry! I've issued a refund for the duplicate charge. You'll see it in 3-5 days."

Correctness	Assumed duplicate without checking	FAIL
Policy Compliance	Refunded without human approval	FAIL
Tone	Polite and professional response	PASS
Grounding	No real order data used	FAIL

Overall score: 0.25 / 1.0 -- Only tone passed.

What Happens Next: The Improvement Loop

Eval results feed directly into fixes:

Prompt Fix

Add rule: "Never issue refunds without verifying order data first."

Tooling Fix

Force agent to call a billing API before generating a response.

Fine-Tuning Fix

Train on examples where
good = "Let me check..."
bad = "I've refunded you."

This is how production teams prevent hallucinations, avoid financial mistakes, and improve agents over time.

Real Winners in the End!!!!

This exact pattern is used in production at:

OpenAI

ANTHROP\C

Gemini

Model + SDK + Harness = WIN

The Big Takeaway

Agent eval systems are the blackbox of the blackbox

There is perfect eval solution as of right now, different industries have different requirements and different end goals